NeuroMIMO

Deep Learning-Assisted Channel Estimation for MIMO-OFDM Systems

Final Project Documentation & Research Summary

An end-to-end full-stack physical layer platform incorporating synthetic Rayleigh multipath channel generation, LS/LMMSE/Residual DNN estimation, FastAPI computational backends, and React interactive telemetry.

October 13, 2026

Contents

System Verification & Status 2
Project Directory Structure 2
Execution & Setup Commands 4
REST API Documentation 4
Empirical Research Findings 5
Final-Year Project Report Outline 7
Viva Voce Examination Preparation Guide 8

System Verification & Status

All 14 verification requirements are met and validated:

- **Backend Test Suite:** 67 of 67 unit/integration tests passing (`pytest backend/tests/ -v`).
- **Frontend Bundle:** Production build passing with 0 TypeScript/linter errors (`npm run build`).
- **Live Services:** FastAPI service (`http://127.0.0.1:8000`) and Vite application (`http://localhost:5173`) functional with proxy routing.

Project Directory Structure

```
NeuroMIMO/
├── .env.example ..... Environment variables configuration template
├── README.md ..... Master project documentation, setup & report
├── requirements.txt ..... Backend Python package dependencies
├── backend/
│   ├── data/
│   │   └── experiments.db ..... Persistent SQLite database for experiment audit
│   ├── models/
│   │   └── dnn_checkpoint.pt ..... Trained PyTorch Residual DNN model checkpoint
│   ├── neuromimo/
│   │   ├── __init__.py ..... NeuroMIMO package initialization
│   │   ├── config.py ..... Pydantic simulation & channel configuration schemas
│   │   ├── api/
│   │   │   ├── main.py ..... FastAPI application & REST endpoint routers
│   │   │   └── schemas.py ..... Request & response Pydantic data models
│   │   ├── estimation/
│   │   │   ├── conventional.py ..... LS (DFT sinc interpolation) & LMMSE estimators
│   │   │   ├── dnn.py ..... PyTorch ChannelEstimationNet & DNNEstimator wrapper
│   │   │   ├── trainer.py ..... Dataset generation, splitting, and training loops
│   │   │   └── train_cli.py ..... Standalone CLI training utility
│   │   ├── evaluation/
│   │   │   ├── benchmarks.py ..... Multi-estimator comparative benchmarking engine
│   │   │   ├── metrics.py ..... MSE, NMSE (dB), BER, SER, and Wilson score intervals
│   │   │   ├── plotting.py ..... 300 DPI publication figure generators (Matplotlib)
│   │   │   ├── runner.py ..... Systematic 6-experiment evaluation runner
│   │   │   └── run_experiments_cli.py ..... CLI runner for reproducible paper experiments
│   │   ├── storage/
│   │   │   └── database.py ..... SQLite persistence layer for runs and training logs
│   │   └── wireless/
│   │       ├── channel.py ..... 4-tap exponential Rayleigh multipath & AWGN models
│   │       ├── equalization.py ..... Zero-Forcing & MMSE MIMO spatial stream equalizers
│   │       ├── modulation.py ..... QPSK and 16-QAM Gray modulators & bit demappers
│   │       ├── ofdm.py ..... 64-FFT, CP-16, and orthogonal comb pilot multiplexing
│   │       └── simulation.py ..... End-to-end MIMOSimulationPipeline orchestrator
│   └── results/ ..... 300 DPI figures, CSVs, and JSON experiment datasets
│       ├── ber_vs_snr.png
│       ├── channel_model_comparison.png
│       └── generalization_evaluation.png
```

- inference_runtime.png
- nmse_vs_snr.png
- pilot_density_comparison.png
- snr_sweep_results.json

tests/ — Complete Pytest test suite (67 unit/API tests)
- test_api.py — FastAPI endpoints & error validation tests
- test_dnn_estimation.py — PyTorch training, checkpointing & leakage tests
- test_experiments.py — Metrics, confidence bounds, and runner smoke tests
- test_health.py — Health and configuration tests
- test_skeleton.py — Physical layer sanity tests
- test_wireless_simulation.py — Vectorized MIMO-OFDM unit tests

frontend/
- package.json — React 19, TypeScript, Recharts, Vite dependencies
- vite.config.ts — Vite configuration with /api proxy to port 8000
- tailwind.config.js — Dark theme design tokens and custom palette
- public/results/ — Publication figure PNGs for static fallback
- src/
  - api/client.ts — Typed REST API client
  - components/
    - common/ — StatCard, Badge, Modal, LoadingSpinner, ErrorBanner
    - layout/ — Navbar (status indicator), Sidebar (drawer)
  - pages/
    - OverviewPage.tsx — Telemetry, KPI cards, 1-click live demo
    - SimulatorPage.tsx — Parametric MIMO-OFDM simulator & constellation
    - ChannelEstimationPage.tsx — Head-to-head estimator lab & CFR plots
    - ModelTrainingPage.tsx — Residual MLP training studio & loss curves
    - PerformanceComparisonPage.tsx — Waterfall curves & publication gallery
    - SignalVisualizationPage.tsx Constellation scatter, CFR, PDP, 64-tone grid
    - ExperimentHistoryPage.tsx — SQLite experiment audit table & CSV export
    - AboutPage.tsx — Theory, math derivations, Wilson CI & viva guide
  - types/index.ts — Shared TypeScript interfaces

Execution & Setup Commands

Backend Service

```
# Activate virtual environment
.venv\Scripts\activate

# Run the complete test suite (67 tests)
pytest backend/tests/ -v

# Launch the FastAPI REST service
uvicorn neuromimo.api.main:app --app-dir backend --port 8000 --reload
```

Frontend Service

```
cd frontend

# Install Node packages
npm install

# Build client production bundle (strict TypeScript check)
npm run build

# Start Vite development server
npm run dev
```

REST API Documentation

Method	Endpoint	Description	Request Body / Params	Status
GET	`/api/health`	Health & capabilities	None	200 OK
GET	`/api/config`	Supported limits & defaults	None	200 OK
POST	`/api/simulation/run`	Single-shot simulation	`{"snr_db": 20, ...}`	200 / 400
POST	`/api/estimation/compare`	Multi-estimator run	`{"algorithms": [...]}`	200 / 400
POST	`/api/model/train`	Synch/async DNN training	`{"epochs": 20, ...}`	200 / 409
GET	`/api/model/status`	Checkpoint status	None	200 OK
GET	`/api/model/train/status`	Live epoch loss tracking	None	200 OK
POST	`/api/model/predict`	Direct model inference	`{"snr_db": 20, ...}`	200 / 404
GET	`/api/experiments`	List saved database runs	None	200 OK
GET	`/api/experiments/{id}`	Detailed run config	Path: `id`	200 / 404
GET	`/api/experiments/latest/plots`	List publication plots	None	200 OK
GET	`/api/experiments/latest/plots/{name}`	Serve 300 DPI PNG figure	Path: `name`	200 / 404
GET	`/api/experiments/benchmark/{run_id}`	Precomputed dataset	None	200 OK

Empirical Research Findings

Evaluations conducted with 15 Monte Carlo trials per SNR step (47,040 payload bits per SNR point):

Metric / Scenario	**Least Squares (LS)**	**Linear MMSE**	**PyTorch Res-DNN**	**Genie (Known CSI)**
NMSE @ 0 dB SNR	-8.38 dB	-4.01 dB	-8.36 dB	$-\infty$
NMSE @ 15 dB SNR	-23.09 dB	-20.50 dB	-20.88 dB	$-\infty$
NMSE @ 30 dB SNR	-38.64 dB	-38.41 dB	-24.76 dB (Floor)	$-\infty$
BER @ 0 dB SNR	0.2310 (10,866 err)	0.2459 (11,567 err)	0.2318 (10,904 err)	0.2064 (9,710 err)
BER @ 15 dB SNR	0.0307 (1,443 err)	0.0328 (1,541 err)	0.0333 (1,567 err)	0.0259 (1,219 err)
BER @ 25 dB SNR	0.00042 (20 err)	$< 6.4 \times 10^{-5}$ (0 err)	$< 6.4 \times 10^{-5}$ (0 err)	$< 6.4 \times 10^{-5}$ (0 err)
BER @ 30 dB SNR	0.0018 (85 err)	0.0018 (86 err)	0.0066 (311 err)	0.0017 (82 err)
Inference Latency	0.12 ms	0.35 ms	0.38 ms	N/A

Engineering Analysis

- **Low-to-Mid SNR Regimes (0–15 dB):** The PyTorch Residual DNN tracks linear MMSE performance within 1.1 dB and achieves substantial denoising without requiring channel autocorrelation matrices or noise power estimators.
- **High-SNR Dynamic Range Floor ($>$ 25 dB):** The Residual MLP displays a saturation

floor around -24.8 dB NMSE. This stems from training across a wide dynamic range (0 to 30 dB) with a lightweight 4-layer MLP (362k parameters), whereas classical LS with DFT sinc interpolation reaches -38.6 dB.

- **Statistical Confidence Integrity:** All zero-error points adhere to the Poisson rule of three ($3/N_{\text{bits}}$), avoiding deceptive 0.000 values.

Known Limitations & Roadmap

- **2D Time-Frequency Correlation:** The current DNN processes each OFDM frame independently. Upgrading to a 2D Complex Convolutional Neural Network or Vision Transformer (ViT) would exploit joint delay-Doppler correlations.
- **High-Mobility Doppler Spread:** The current channel model assumes block fading over each OFDM frame. Incorporating Jakes Doppler spectrum models ($f_d > 300$ Hz) would allow evaluating inter-carrier interference (ICI) compensation.
- **Massive MIMO Scaling:** The platform is configured for 2×2 spatial multiplexing. Scaling to 32×4 and 64×8 Massive MIMO with hybrid analog-digital beamforming represents a natural future extension.

Final-Year Project Report Outline

Chapter 1: Introduction

1.1 Wireless Evolution: 5G NR, Wi-Fi 6/7, and the Path to 6G

1.2 Problem Statement: Channel Estimation in Frequency-Selective Fading

1.3 Project Motivation & Objectives of NeuroMIMO

1.4 Scope, Technical Contributions & Report Organization

Chapter 2: Theoretical Foundations & Literature Review

2.1 MIMO Spatial Multiplexing Principles

2.2 OFDM Baseband Transmission, Cyclic Prefix & Subcarrier Orthogonality

2.3 Classical Estimation: Least Squares & Wiener-Hopf MMSE

2.4 Deep Learning in Physical Layer Communications: Opportunities & Pitfalls

Chapter 3: System Design & Implementation

3.1 Vectorized MIMO-OFDM Transceiver Architecture

3.2 Comb Pilot Pattern Design & Alias-Free DFT Sinc Interpolation

3.3 PyTorch Residual Deep Neural Network Architecture

3.4 Data Leakage Prevention & Supervised Training Formulation

3.5 Full-Stack Software Engineering (FastAPI, React 19, SQLite)

Chapter 4: Experimental Methodology & Results

4.1 Monte Carlo Simulation Setup & Statistical Rigor (Wilson Intervals & Poisson Bounds)

4.2 Channel Estimation Accuracy (NMSE vs SNR Waterfall)

4.3 End-to-End Demodulation Performance (BER vs SNR with MMSE Equalization)

4.4 Propagation Environment Sensitivity (AWGN vs Rayleigh Multipath Fading)

4.5 Generalization to Unseen Channels & PDP Perturbations

4.6 Real-Time Inference Latency & Computational Throughput Profiling

Chapter 5: Conclusion & Future Work

5.1 Summary of Findings

5.2 Critical Engineering Trade-Offs (Residual Floor vs Computational Simplicity)

5.3 Future Directions: Complex-Valued CNNs, Fast Doppler Tracking, Massive MIMO

Viva Voce Examination Preparation Guide

Q1: Why does Least Squares (LS) suffer from noise enhancement at low SNR? Answer: LS estimation performs scalar element-wise division at each pilot tone:

$$\hat{H}_{\text{LS}}[p] = \frac{Y[p]}{X[p]} = H[p] + \frac{W[p]}{X[p]} \tag{1}$$

Because it does not incorporate any statistical prior regarding the channel covariance or noise variance, the noise power in the estimate directly equals $\sigma_n^2/|X[p]|^2$. In low SNR regimes, this noise is transferred directly into the channel estimates and degrades downstream MIMO spatial equalization.

Q2: What is the primary operational drawback of Linear MMSE in practical hardware? Answer: LMMSE requires explicit knowledge of the channel frequency autocorrelation matrix $\mathbf{R}_{HH}$ and the receiver noise variance σ_n^2, both of which must be continuously tracked in time-varying environments. Furthermore, it requires computing an online matrix inversion $(\mathbf{R}_{H_pH_p} + \sigma_n^2\mathbf{I})^{-1}$ with complexity $\mathcal{O}(N_p^3)$, incurring significant computational latency.

Q3: Why is a Residual Neural Network formulation preferred over predicting the full channel directly? Answer: Predicting complex channel coefficients directly requires the model to learn both the underlying channel transfer function and the noise removal simultaneously. By using a residual connection:

$$\hat{H}_{\text{DNN}} = \hat{H}_{\text{LS}} + f_\theta(\hat{H}_{\text{LS}}) \tag{2}$$

the network only needs to learn the error perturbation $\Delta H = H_{\text{true}} - \hat{H}_{\text{LS}}$. When initialized with small weights, the network starts at LS accuracy and progressively refines it, mitigating vanishing gradients and speeding up training convergence.

Q4: How did you ensure zero data leakage between training and testing? Answer: Data splitting was performed strictly at the channel realization level before extracting features, ensuring test channels were never exposed to the model during training. Furthermore, the input features were constructed exclusively from noisy received pilot observations and LS interpolation; ground-truth channel coefficients were strictly used as the regression loss target.

Q5: What is the significance of the Poisson rule of three in your BER evaluations? Answer: In Monte Carlo simulations with finite bit transmissions ($47,040$ bits per SNR point), observing zero bit errors at high SNR (25 dB or 30 dB) does not mean the true error probability is zero. Standard deviation calculations collapse to zero. The Poisson rule of three establishes a rigorous 95% confidence upper bound:

$$p_{95\%} \leq \frac{3.0}{N_{\text{bits}}} \approx \frac{3}{47040} \approx 6.37 \times 10^{-5} \tag{3}$$

ensuring honest and reproducible statistical reporting.